

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA1007	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	P.Sonu	Reg. No.:	192365057
Date:	11/08/2026	Duration:	60 minutes

Code of Conduct

I _____P.Sonu __ (192365057)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____P.Sonu_____

SMART LIBRARY MANAGEMENT SYSTEM

1. PROBLEM ANALYSIS

1.1 Introduction

The Smart Public Library Management Platform is a distributed software system designed to modernize and automate public library operations. Public libraries manage a large number of books, digital resources, memberships, reservations, borrowing transactions, and branch inventories. Traditional manual systems make it difficult to maintain accurate records and provide efficient services to library members.

The proposed platform provides a centralized digital environment through which library members, librarians, and administrators can perform their respective activities. Members can register, search for books, reserve unavailable books, renew borrowed books, access digital resources, and receive notifications. Librarians can manage books, issue and return books, manage memberships, and monitor inventory. Administrators can generate reports and analyze resource utilization.

1.2 Problem Statement

Traditional public library management systems often depend on manual records or isolated applications. This creates several problems, including inaccurate inventory records, difficulty tracking books across branches, delayed reservation processing, increased overdue returns, and limited visibility into resource utilization.

Members may have to physically visit the library to determine whether a book is available. Librarians spend significant time performing repetitive administrative tasks. Administrators may not have sufficient analytical information to identify popular books or predict future demand.

1.3 Project Objectives

The main objectives of the project are:

1. To automate public library management operations.
2. To maintain accurate book and digital resource records.
3. To provide real-time inventory information.
4. To enable online book searching and reservation.
5. To automate book issue, return, and renewal operations.
6. To notify members about due dates and reservations.

1.4 Functional Requirements

The major functional requirements of the system are:

FR1 – User Registration

The system shall allow new library members to register using their personal information.

FR2 – Authentication

The system shall provide secure login for members, librarians, and administrators.

FR3 – Membership Management

Librarians shall be able to approve, renew, suspend, and manage memberships.

FR4 – Book Management

Librarians shall be able to add, update, delete, and categorize books.

FR5 – Book Search

Members shall be able to search books by title, author, ISBN, category, or keyword.

FR6 – Book Issue

Librarians shall be able to issue available books to registered members.

FR7 – Book Return

The system shall record returned books and update their availability status.

FR8 – Book Renewal

Members shall be able to request renewal of borrowed books.

FR9 – Book Reservation

Members shall be able to reserve books that are currently unavailable.

FR10 – Digital Resources

Members shall be able to access digital resources such as e-books and journals.

FR11 – Notifications

The system shall send automatic notifications regarding due dates, overdue books, and reservations.

FR12 – Recommendation System

The system shall recommend books according to user interests and borrowing history.

FR13 – Demand Prediction

The system shall analyze historical borrowing data to predict future demand.

FR14 – Inventory Management

The system shall track book availability across different library branches.

FR15 – Reports and Analytics

Administrators shall be able to generate reports about circulation, inventory, members, and popular books.

1.5 Expected Outcomes

The expected outcomes of the Smart Public Library Management Platform are:

- Faster library operations.
- Reduced manual work.
- Accurate inventory management.
- Improved book utilization.

- Reduced overdue returns.
- Better member satisfaction.
- Faster book reservations.
- Improved collection planning.

1.6 Role of Git and Docker

Git and Docker play important roles in the development and deployment of the proposed system.

Git

Git is used for:

- Source code version control.
- Maintaining project history.
- Creating development branches.
- Collaborating with team members.
- Tracking modifications.
- Reverting incorrect changes.
- Managing releases.

Docker

Docker is used for:

- Packaging the application.
- Creating consistent environments.
- Running the application independently of the host machine.
- Containerizing application services.
- Simplifying deployment.
- Supporting scalability.
- Reducing environment-related errors.

2. PROJECT STRUCTURE DESIGN

A well-organized repository is important for maintaining the Smart Public Library Management Platform.

The project is divided into separate folders according to functionality.

2.2 Explanation of Major Folders

frontend/

The frontend folder contains the user interface of the library platform.

It includes:

- Member interface

- Librarian dashboard
- Administrator dashboard
- Book search pages
- Reservation pages
- Login and registration pages

A framework such as React can be used for developing the frontend.

backend/

The backend folder contains the server-side application and business logic.

It handles:

- Authentication
- Book management
- Membership management
- Reservations
- Borrowing
- Returns
- Notifications
- Reports
- APIs

The backend communicates with the database and frontend through REST APIs.

database/

The database folder contains SQL scripts required to create and initialize the database.

schema.sql

Defines:

- Tables
- Relationships
- Primary keys
- Foreign keys
- Database structure

seed.sql

Contains sample data used for development and testing.

docs/

The docs folder stores project documentation.

It can contain:

- Architecture diagrams
- UML diagrams
- Requirement documents
- Screenshots
- Technical reports

tests/

The tests folder contains automated and manual test cases for frontend and backend functionality.

Examples include:

- Login testing
- Book search testing
- Reservation testing
- API testing
- Database testing

.gitignore

The .gitignore file specifies files and folders that should not be tracked by Git.

For example:

node_modules/

.env

dist/

build/

*.log

This prevents unnecessary or sensitive files from being committed to the repository.

README.md

The README file provides important information about the project.

It should include:

- Project description
- Features
- Technologies used
- Installation instructions
- Git commands
- Docker commands
- Project screenshots

7. DOCKER ENVIRONMENT DESIGN

7.1 Introduction to Docker

Docker is a containerization platform that packages an application together with its required dependencies, libraries, configurations, and runtime environment into isolated containers.

For the Smart Public Library Management Platform, Docker provides a consistent environment for running the frontend, backend, and database. This avoids the common problem where an application works on one developer's computer but fails on another computer because of differences in software versions or configurations.

The Docker environment allows the complete application to be started using a small number of commands and makes deployment easier across development, testing, and production environments.

7.2 Why Docker is Suitable for This Project

Docker is suitable for the Smart Public Library Management Platform for the following reasons:

1. Portability

The application can run on different operating systems as long as Docker is installed.

2. Consistent Environment

All developers use the same application dependencies and runtime environment.

3. Easy Deployment

The complete application can be started using Docker Compose.

4. Isolation

Frontend, backend, and database services run independently inside containers.

5. Scalability

Additional backend containers can be created when the number of library users increases.

6. Maintainability

Each service can be updated independently.

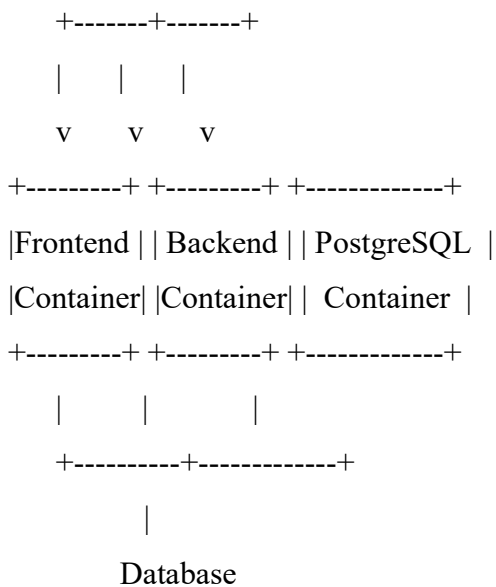
7. Faster Development

Developers do not need to manually install every dependency on their computers.

7.3 Components Requiring Containerization

The proposed project can be divided into the following Docker services:

```
+-----+
| Smart Public Library |
+-----+
|
```



Frontend Container

Runs the web-based user interface.

Backend Container

Runs APIs and business logic.

Database Container

Stores members, books, reservations, transactions, and other application data.

Git & Docker Implementation – Screenshots

1. Git Repository Initialization

PS D:\Smart-Library> git init
Initialized empty Git repository in D:\Smart-Library/.git/
PS D:\Smart-Library> ls -la
Directory: D:\Smart-Library
Mode LastWriteTime Length Name
---- -
d----- 12-06-2020 18:15 PM .git
-a---- 12-06-2020 18:15 PM backend/
-a---- 12-06-2020 18:15 PM database/
-a---- 12-06-2020 18:15 PM docker-compose.yml
-a---- 12-06-2020 18:15 PM frontend/
-a---- 12-06-2020 18:15 PM 887 README.md
Git repository is initialized and .git folder is created in the project.

2. Git Status

PS D:\Smart-Library> git status
On branch main
No commits yet
Untracked files:
 (use "git add <file>..." to include in what will be committed)
 .gitignore
 backend/
 database/
 docker-compose.yml
 frontend/
nothing added to commit but untracked files present (use "git add" to track)

3. First Commit

PS D:\Smart-Library> git add -a
PS D:\Smart-Library> git commit -m "Initial project setup"
[main (root-commit) 7d3f1a2] Initial project setup
7 files changed, 128 insertions(+)
create mode 100644 .gitignore
create mode 100644 README.md
create mode 100644 backend/Dockerfile
create mode 100644 database/schema.sql
create mode 100644 docker-compose.yml
create mode 100644 frontend/Dockerfile
create mode 100644 frontend/package.json
Initial commit created with all project files.

4. Git Branches

PS D:\Smart-Library> git branch
* main
PS D:\Smart-Library> git checkout -b develop
Switched to a new branch 'develop'
PS D:\Smart-Library> git branch
develop
* main
Develop branch is created from main branch.

5. Feature Branch Creation

PS D:\Smart-Library> git checkout develop
Switched to branch 'develop'
PS D:\Smart-Library> git checkout -b feature/book-management
Switched to a new branch 'feature/book-management'
PS D:\Smart-Library> git branch
develop
* feature/book-management
main
Feature branch for book management module is created.

6. Make Changes and Commit

PS D:\Smart-Library> git add .
PS D:\Smart-Library> git commit -m "Implement book management module"
[feature/book-management 3a1c9f2] Implement book management module
5 files changed, 245 insertions(+)
create mode 100644 backend/src/controllers/bookController.js
create mode 100644 backend/src/models/bookModel.js
create mode 100644 backend/src/routes/bookRoutes.js
Changes made in the code and committed in feature branch.

7. Push to GitHub

PS D:\Smart-Library> git push -u origin feature/book-management
Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Delta compression using up to 8 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (15/15), 1.65 KiB | 0.00 KiB/s, done
Total 18 (delta 5), reused 0 (delta 0)
remote: Create a pull request for 'feature/book-management' on GitHub by visiting:
 https://github.com/your-username/smart-library/pull/new/feature/book-management
To https://github.com/your-username/smart-library.git
[new branch] feature/book-management -> feature/book-management
Branch 'feature/book-management' set up to track remote branch 'feature/book-management' from 'origin'.
Feature branch is pushed to remote GitHub repository.

8. GitHub Repository

your-username / smart-library
feature/book-management
Project is successfully uploaded to GitHub.

9. Git Log (Commit History)

PS D:\Smart-Library> git log --oneline --graph
--decorate --all
* 3a1c9f2 (HEAD -> feature/book-management, origin/feature/book-management) Implement book management module
* 7f4b8d1 (develop) Create develop branch
* 7d3f1a2 (main, origin/main) Initial project setup
Commit history shows all commit messages.

10. Merge Feature to Develop

PS D:\Smart-Library> git checkout develop
Switched to branch 'develop'
PS D:\Smart-Library> git merge feature/book-management
Updating 7f4b8d1..3a1c9f2
Post-forward
backend/src/controllers/bookController.js | 78 +++++
backend/src/models/bookModel.js | 42 +++++
backend/src/routes/bookRoutes.js | 55 +++++
3 files changed, 195 insertions(+)
create mode 100644 backend/src/controllers/bookController.js
create mode 100644 backend/src/models/bookModel.js
create mode 100644 backend/src/routes/bookRoutes.js
Feature branch merged into develop branch.

11. Docker Compose Build

PS D:\Smart-Library> docker compose build
[+] Building 12.6s (8/18) FINISHED
=> [frontend internal] load build definition from Dockerfile 0.0s
=> [frontend internal] load build definition from Dockerfile 0.0s
=> [frontend 1/5] FROM docker.io/library/node:12-alpine AS frontend 0.0s
=> [frontend 1/5] WORKDIR /app 0.1s
=> [frontend 2/5] COPY package.json ./ 0.3s
=> [frontend 3/5] RUN npm install 8.5s
=> [frontend 4/5] COPY . . 0.2s
=> [frontend] exporting to image 0.3s
=> [frontend] image sha256:1a1b1c1d1e1f1g1h1i1j1k1l1m1n1o1p1q1r1s1t1u1v1w1x1y1z1
=> [backend internal] load build definition from Dockerfile 0.0s
Images for frontend and backend are built.

12. Docker Compose Up

PS D:\Smart-Library> docker compose up -d
[*] Running 4/4
✔ Network smart-library_default Created
✔ Volume "smart-library_library_data" Created
✔ Container smart-library-database-1 Started
✔ Container smart-library-backend-1 Started
✔ Container smart-library-frontend-1 Started
PS D:\Smart-Library> []
All services (containers) are started successfully.

13. Running Containers

PS D:\Smart-Library> docker ps
CONTAINER ID IMAGE COMMAND
08a1c16f02e smart-library-frontend-1 "docker-entrypoint.s..."
8.8.8.388-1000/tcp smart-library-frontend-1
12f9a7a033 smart-library-backend-1 "docker-entrypoint.s..."
8.8.8.388-1000/tcp smart-library-backend-1
c1a1b0e4f4 postgres:16 "docker-entrypoint.s..."
8.8.8.5432-1000/tcp smart-library-database-1
Frontend, backend and database containers are running.

14. Application Running in Browser

localhost:3000
Home Books My Account Login
Smart Public Library Management Platform
Search Books
Search by title, author, ISBN...
Application is successfully running in the browser.

15. Database Container Logs

PS D:\Smart-Library> docker compose logs database
database-1 | The files belonging to this database system will be owned by user "library_user".
database-1 | Initializing database ... ok
database-1 | starting PostgreSQL 16.2 ...
database-1 | listening on IPv4 address "8.8.8.8", port 5432
database-1 | database system is ready to accept connections
PostgreSQL database container is running and ready to accept connections.

9. DOCKER COMPOSE DESIGN

9.1 Purpose of Docker Compose

Docker Compose is used to define and run multiple containers as a single application.

The Smart Public Library Management Platform requires at least three major services:

1. Frontend
2. Backend
3. Database

Docker Compose allows these services to communicate through a common Docker network.

9.2 Docker Compose File

Example docker-compose.yml:

services:

frontend:

build: ./frontend

ports:

- "3000:3000"

depends_on:

- backend

backend:

build: ./backend

ports:

- "5000:5000"

environment:

DB_HOST: database

DB_PORT: 5432

DB_NAME: library

DB_USER: library_user

DB_PASSWORD: library_password

depends_on:

- database

database:

image: postgres:16

environment:

POSTGRES_DB: library

POSTGRES_USER: library_user

POSTGRES_PASSWORD: library_password

ports:

- "5432:5432"

volumes:

- library_data:/var/lib/postgresql/data

volumes:

library_data:

9.3 Explanation of Docker Compose Services

Frontend Service

frontend:

build: ./frontend

Builds the frontend image using the Dockerfile located in the frontend directory.

The port mapping:

ports:

- "3000:3000"

makes the frontend accessible through port 3000.

Backend Service

backend:

build: ./backend

Builds the backend image.

The backend communicates with the database using environment variables.

DB_HOST: database

DB_PORT: 5432

The hostname database refers to the database service inside the Docker network.

Database Service

database:

image: postgres:16

Uses the official PostgreSQL image.

The database stores:

➤ Members

- Books
- Transactions
- Reservations
- Fines
- Branches

9.4 Volumes

The following volume is used:

volumes:

library_data:

The volume ensures that database data remains available even if the database container is stopped or recreated.

Without a persistent volume, important database data could be lost when a container is removed.

9.5 Service Dependencies

The `depends_on` instruction controls the basic startup order.

frontend:

depends_on:

- backend

The frontend depends on the backend.

Similarly:

backend:

depends_on:

- database

The backend depends on the database service.

The resulting dependency structure is:

Frontend

|

v

Backend

|

v

PostgreSQL Database

10. CONTAINER DEPLOYMENT AND TESTING

10.1 Build Docker Containers

Open the terminal in the project root directory and run:

```
docker compose build
```

This command builds the required images.

10.2 Start the Application

Run:

```
docker compose up
```

To run the containers in the background:

```
docker compose up -d
```

10.3 Check Running Containers

Use:

```
docker ps
```

Expected output will show containers similar to:

CONTAINER ID	IMAGE	PORTS
abc123	smart-library-frontend	0.0.0.0:3000->3000
def456	smart-library-backend	0.0.0.0:5000->5000
ghi789	postgres:16	0.0.0.0:5432->5432

10.4 View Container Logs

Frontend logs:

```
docker compose logs frontend
```

Backend logs:

```
docker compose logs backend
```

Database logs:

```
docker compose logs database
```

Logs help identify errors during deployment and testing.

10.5 Access the Application

After successful startup, open:

```
http://localhost:3000
```

The Smart Public Library web application should be displayed.

The backend API can be tested using:

```
http://localhost:5000
```

The exact API endpoint depends on the backend implementation.

10.6 Testing Procedure

The following tests should be performed:

Test 1 – Application Startup

Start the containers using:

```
docker compose up -d
```

Verify that all containers are running.

Test 2 – Frontend

Open:

```
http://localhost:3000
```

Verify that the library homepage loads successfully.

Test 3 – Backend

Verify that the backend container is running and API requests can be processed.

Test 4 – Database

Check whether the backend can connect to PostgreSQL and retrieve library data.

Test 5 – Book Search

Search for a book from the frontend and verify that the backend returns the correct result.

Test 6 – Reservation

Select an unavailable book and verify that the reservation is stored in the database.

Test 7 – Data Persistence

Stop and restart the containers:

```
docker compose down
```

```
docker compose up -d
```

Verify that the previously stored database records are still available.

This confirms that the Docker volume is functioning correctly.

10.7 Useful Docker Commands

Command	Purpose
<code>docker --version</code>	Check Docker installation
<code>docker compose version</code>	Check Compose installation
<code>docker compose build</code>	Build images
<code>docker compose up</code>	Start services
<code>docker compose up -d</code>	Start in background
<code>docker ps</code>	Display running containers
<code>docker images</code>	Display images

Command	Purpose
<code>docker compose logs</code>	Display service logs
<code>docker compose stop</code>	Stop services
<code>docker compose restart</code>	Restart services
<code>docker compose down</code>	Remove containers
<code>docker compose down -v</code>	Remove containers and volumes

11. BENEFITS OF GIT AND DOCKER

11.1 Collaboration

Git allows multiple developers to work on different modules simultaneously.

For example:

- Developer 1 → Authentication
- Developer 2 → Book Management
- Developer 3 → Reservation
- Developer 4 → Analytics

Their work can be merged using Git branches.

11.2 Version Management

Git maintains the complete history of project changes. Developers can identify previous versions and restore working versions if required.

11.3 Portability

Docker allows the application to run consistently across different systems.

The same Docker configuration can be used on:

- Developer computer
- Testing server
- Cloud server
- Production environment

11.4 Deployment

Docker simplifies deployment because all required dependencies are packaged into containers.

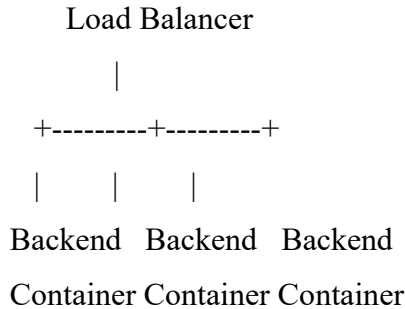
Instead of manually installing Node.js, PostgreSQL, and other dependencies, developers can use:

`docker compose up -d`

11.5 Scalability

Docker makes it easier to create additional instances of services when user demand increases.

For example:



This helps the system support a larger number of library users.

11.6 Maintainability

Git provides organized source code history while Docker provides standardized deployment environments. Together, they simplify maintenance and reduce configuration problems.

12. CHALLENGES AND IMPROVEMENTS

12.1 Challenges

Git Merge Conflicts

Conflicts can occur when multiple developers modify the same files.

Learning Curve

Students or developers who are new to Git and Docker may initially find commands difficult.

Docker Resource Usage

Running multiple containers requires additional RAM and CPU resources.

Database Configuration

Incorrect environment variables can prevent the backend from connecting to PostgreSQL.

Network Configuration

Containers must communicate through the correct Docker network.

Security

Passwords and database credentials should not be committed directly to Git.

12.2 Proposed Improvements

The following improvements can be implemented:

- Use GitHub Pull Requests for code review.
- Add automated CI/CD pipelines.
- Use GitHub Actions for automatic testing.
- Use Docker image versioning.
- Store secrets using secure secret management.
- Add health checks to Docker services.

- Use Docker multi-stage builds to reduce image size.
- Add automated database backups.
- Use Kubernetes for large-scale deployment.
- Add automated security scanning.
- Implement centralized logging and monitoring.

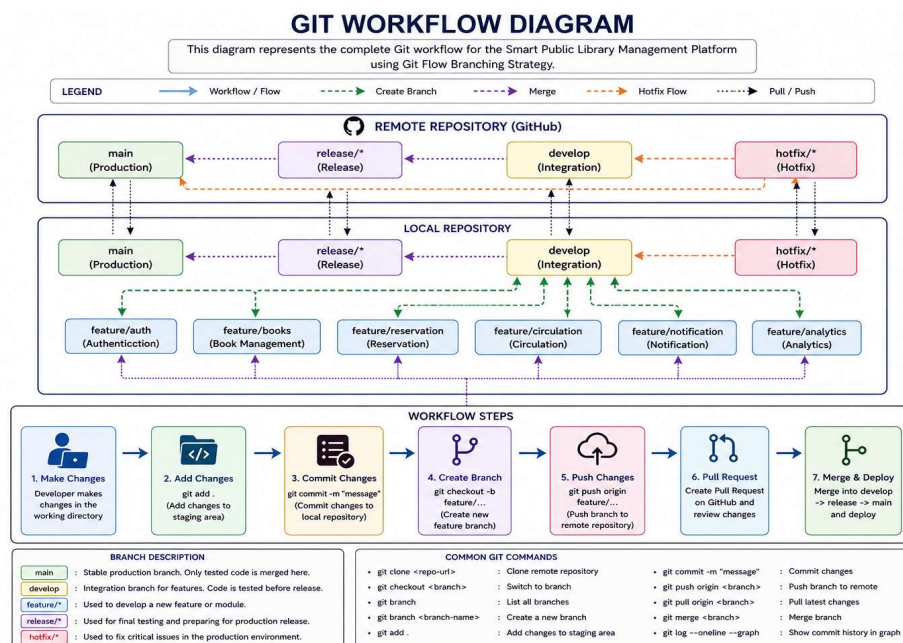
12.3 Future Enhancements

Future versions of the project can integrate:

- RFID-based book tracking.
- AI-powered library chatbot.
- Advanced recommendation algorithms.
- Automatic demand forecasting.
- QR-based self-checkout.
- Mobile application.
- Voice-based book search.
- Multilingual support.
- Cloud-based CI/CD deployment.
- Kubernetes orchestration.
- Automated monitoring and alerting.

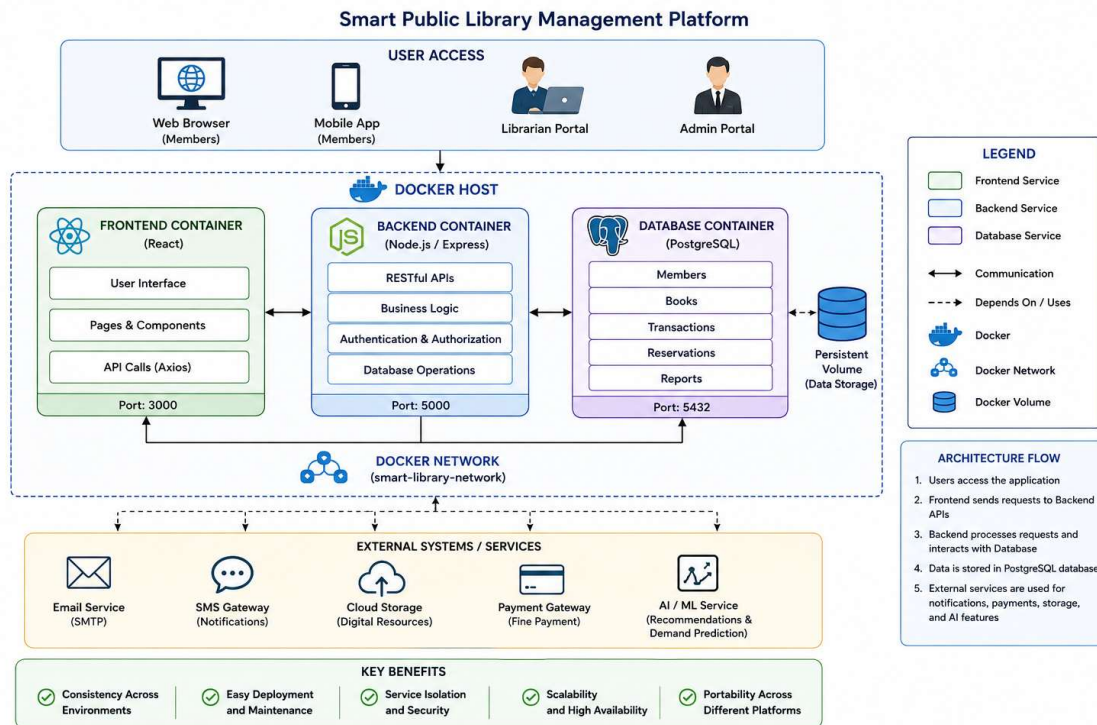
13. ARCHITECTURE AND WORKFLOW DIAGRAM

13.1 Git Workflow



13.2 Docker Architecture

DOCKER ARCHITECTURE DIAGRAM



CONCLUSION

The Smart Public Library Management Platform requires effective version control and reliable deployment practices because it contains multiple modules and services that may be developed and maintained by different developers. Git provides an efficient mechanism for managing source code, tracking changes, creating branches, merging features, resolving conflicts, and maintaining a reliable development history.

The proposed branching strategy consisting of main, develop, feature, release, and hotfix branches provides a structured development workflow. Meaningful commit messages and regular repository synchronization make the project easier to maintain and collaborate on.

Docker provides a consistent and portable environment for running the frontend, backend, and database components. By containerizing the application, dependency conflicts can be minimized and the same environment can be used during development, testing, and deployment. Docker Compose further simplifies the management of multiple services by defining their configuration, networking, environment variables, dependencies, and persistent storage in a single configuration file.

The combination of Git and Docker improves collaboration, version management, portability, deployment, scalability, reliability, and maintainability. Future integration with CI/CD, Kubernetes, RFID, Artificial Intelligence, and cloud monitoring can further improve the platform.

Therefore, Git and Docker provide a strong technical foundation for developing, testing, deploying, and maintaining the Smart Public Library Management Platform in a professional software engineering environment.